

Decision-OS V7 Final: Guardable Self-Recursive Evolution

Structural Failure Boundaries for Self-Recursive Intelligence

Shinichi Nagata

Abstract

Decision-OS is a gate-first operational architecture for preserving guardable continuity across decision, update, integration, memory, release, and completion. This paper applies that architecture to self-recursively updating intelligence and defines the structural failure boundaries of guardable self-recursive evolution.

This paper does not attempt to fully define AGI. Nor does it prove sufficient conditions for the occurrence of self-evolution. Its target is narrower: to define the structural failure modes that any account of self-recursive intelligence must address before it can claim guardable self-recursive evolution. These failure modes include vacuous recursion, drift, rupture, self-justifying closure, hindsight rewriting, memory discontinuity, carrier collapse, and false completion.

The central thesis is the Guardable Continuity Axiom: evolution is admissible only when change preserves guardable continuity. Under this axiom, Aspire, Guard, and Intelligence as self-recursion on PIC are treated not as numerical factors but as a logical meet. Aspire supplies direction, Guard supplies admissibility boundaries, and Intelligence on PIC supplies the non-destructive substrate for self-recursive updating.

The paper then introduces a C1–C9 Failure Mode Coverage Map. These conditions are not sufficient conditions for AGI. Instead, they identify which known failure modes reopen when each condition is removed. Finally, the paper introduces Reconstructive Delta as an observable operational handle for converting reconstruction error into possible material for future self-recursive updating, without treating it as proof of self-evolution.

V7 Final therefore does not declare the arrival of AGI. It defines structural necessary conditions and failure boundaries that self-recursive intelligence must address before claiming evolvability.

Index Terms

Decision-OS, AGI definition, Aspire, Guard, Self-Recursion, PIC, Guardable Continuity, Failure Mode Coverage, External Anchor, Reconstructive Delta

I. Introduction

Artificial general intelligence is often discussed in terms of capability. A system is called general when it solves many tasks, approaches human-level performance, generalizes to unfamiliar environments, or exceeds existing benchmarks. These observations are important, but they do not provide a stable structural definition. Benchmarks change, evaluation environments shift, leakage and distributional effects are difficult to remove completely, and deployment contexts alter what a reported score means. If AGI is defined only by capability, it becomes a moving performance target rather than a fixed structural condition.

This paper approaches the problem from another direction. V7 Final does not attempt to fully define AGI. Instead, it defines the structural failure modes that any account of self-recursive intelligence must address. More precisely, this paper asks what conditions are required for a self-recursive system to keep updating without collapsing into unguardable update, drift, rupture, self-justifying closure, hindsight rewriting, memory discontinuity, carrier collapse, or false completion.

This position is not an AGI arrival declaration. It is also not a proof of sufficient conditions for self-evolution. The claim is narrower. If an account of intelligence includes self-recursion, then it must explain how direction is preserved, how boundaries are enforced, how memory and updates are integrated without rupture, how past judgments are not rewritten by hindsight, and how completion remains restartable by a future self.

V7 Final therefore does not present sufficient conditions for AGI. It defines structural necessary conditions and failure boundaries that must be addressed before self-recursive intelligence can claim evolvability.

V7 Final also does not provide a full operational gate implementation. More concrete gates for carrier preservation, reconnectable memory, and future-restartable completion are addressed in later Decision-OS layers such as V10–V12. The purpose of V7 Final is strictly to define the structural failure boundaries that any account of guardable self-recursive evolution must confront.

A. Canonical Definition of Decision-OS

In this paper, Decision-OS is defined as follows:

Decision-OS is a gate-first operational architecture for preserving guardable continuity across decision, update, integration, memory, release, and completion.

TABLE I
Representative Gate Outputs in Decision-OS

Output	Meaning
PASS	The required conditions are satisfied; the process may proceed.
DELAY	Evidence, confirmation, or re-evaluation is insufficient; the process should wait.
BLOCK	Proceeding would create dangerous or irreversible damage; the process must stop.
HOLD	Judgment material is insufficient; the system remains in an observation state.
REVOKE	A previous approval or execution intention is cancelled.
MERGE	Multiple updates are integrated non-destructively.
FORGET	Memory is compressed in a reconnectable form rather than merely deleted.
RELEASE	The artifact or decision is moved into a publishable state.
CLOSE	The work is completed in a form that a future self can restart.

The important point is that Decision-OS is not merely a philosophical operating system. Philosophy can state how a system ought to behave. Decision-OS goes further by treating operational outputs as part of the architecture: when to PASS, DELAY, BLOCK, HOLD, REVOKE, MERGE, FORGET, RELEASE, or CLOSE.

Accordingly, Decision-OS is not an abstract declaration of values. It is an operational structure with gates, states, merge rules, evidence anchors, audit records, and failure modes.

V7 Final extends this gate-first structure to self-recursive intelligence. The relevant question is not whether an intelligence can update itself. The relevant question is whether an updating intelligence can avoid becoming a structure that can no longer be guarded.

B. Central Thesis: The Guardable Continuity Axiom

The central thesis of V7 Final is the following:

Evolution is admissible only when change preserves guardable continuity.

This statement is called the Guardable Continuity Axiom.

Continuity here does not mean preserving the same memory forever. It also does not mean preserving the same output tendency. Continuity means that direction, boundary, update rule, responsibility, memory, and restartability remain reconnectable after change.

Guardable means that change is not unlimited. An update may increase capability, but if it drifts into an uncontrollable region, rewrites past judgments through hindsight, breaks memory continuity, disguises completion, or leaves the future self unable to restart, then that update is not evolution in the sense used by V7 Final.

Recursive updating is not intelligence unless it can evolve without becoming unguardable.

C. Contributions

The contribution of this paper is not to claim that AGI has been solved. Its contribution is to define the minimum set of failure modes that a system claiming self-recursive evolution must resolve, refute, or replace.

First, the paper defines Decision-OS as a gate-first operational architecture for preserving guardable continuity across decision, update, integration, memory, release, and completion.

Second, it introduces the Guardable Continuity Axiom and defines evolution not as capability gain, but as change that preserves guardable continuity.

Third, it interprets Aspire $\times$ Guard $\times$ Intelligence as a logical meet rather than numerical multiplication. Aspire supplies long-horizon direction, Guard supplies admissibility boundaries, and Intelligence is treated as self-recursion on PIC, providing a non-destructive substrate for self-updating.

Fourth, it introduces the C1–C9 Failure Mode Coverage Map. This map does not provide sufficient conditions for AGI. It identifies which known failure modes reopen when each condition is missing.

Fifth, it introduces Reconstructive Delta as an operational handle for treating reconstruction error not merely as failure, but as a detectable difference that may become material for future self-recursive updating.

The purpose of this paper is therefore not to generate self-evolution. Its purpose is to define which structural failures cannot be ignored before self-evolution is claimed.

II. Decision-OS as Gate-First Operational Architecture

Decision-OS is not a name for a thinking style. It is also not a philosophical catalogue of abstract values. In this paper, Decision-OS is a gate-first operational architecture for preserving guardable continuity across decision, update, integration, memory, release, and completion.

TABLE II
Six Domains Covered by Decision-OS

Domain	Question
Decision	What should be chosen? Which action is admissible? Who holds the final seat of responsibility?
Update	What should be changed? Which delta should be accepted?
Integration	How should multiple inputs, evidence sources, memories, or model outputs be joined non-destructively?
Memory	What should be retained, compressed, forgotten, or preserved in reconnectable form?
Release	When should an artifact or decision be exposed externally, and how should As-of responsibility be fixed?
Completion	When should work be closed, and in what form can a future self restart it?

A gate-first architecture does not pass an input, proposal, update, release, or closure directly into execution. It first evaluates whether the candidate operation satisfies the conditions for action, reversibility, responsibility, memory preservation, evidence anchoring, and future restartability.

A. What Gate-First Means

Gate-first means that the system evaluates passage conditions before action, update, integration, release, or completion. In ordinary AI systems, an input is typically followed by an output. In ordinary decision-making, desire, goal, prediction, expected value, social pressure, or intuition often comes first.

Decision-OS places a gate before execution.

A gate passes what may proceed, delays what requires additional evidence or confirmation, blocks what would create unacceptable damage, reconnects what must be preserved, and structures the differences that should be sent forward.

In this sense, Decision-OS does not end at the question of how a system should think. It also specifies which operational state the system should enter.

B. Why the Gate Must Come Before Execution

The most dangerous failure in self-recursive intelligence is not merely lack of capability. As capability increases, a wrong update or false completion can pass more fluently, more naturally, and in a less detectable form.

If a wrong update occurs once, it may be a local error. In a self-recursive system, however, that error can become the premise of the next update. That premise may then be compressed, integrated, and passed into later decisions. A local error can therefore become trajectory drift, memory corrosion, hindsight rewriting, or false completion.

For this reason, output quality is not sufficient for self-recursive systems. What is needed is a gate that evaluates whether an output may enter the next self-update.

Without such a gate, self-recursion tends to fall into at least the following failures:

- recursion without direction;
- uncontrollable drift;
- rupture of prior judgments, evidence, memory, or constraints;
- self-justifying closure;
- false completion.

These are not merely secondary safety risks. They are failures that undermine the definition of self-recursive intelligence itself.

C. Scope of Decision-OS

Decision-OS does not govern only a single decision. In this paper, it spans six domains: decision, update, integration, memory, release, and completion.

The shared problem across these domains is how to handle change without breaking continuity. In a self-recursive system, decisions generate updates; updates require integration; integration changes memory; memory becomes the premise of the next decision; release fixes As-of responsibility; and completion becomes a handoff to the future self.

Optimizing only one domain cannot preserve the whole line. A system may make a locally correct decision while corrupting memory. It may integrate quickly while diluting safety signals. It may release early while erasing As-of responsibility. It may close work while leaving the future self unable to restart.

Decision-OS treats this entire sequence as a gate-first structure.

TABLE III
Decision-OS Lineage as Gate Stack

Version	Role in V7 Final	Failure if Missing
V4	Direction / Structural Orientation	Directionless self-recursion; empty coherence loop
V5	Guard / Irreversible Execution Boundary	Unguardable update; irreversible harm; execution outside Guard
V6	PIC / Non-Destructive Integration	Rupture; order dependence; last-write-wins; safety dilution
V7	Guardable Self-Recursive Admissibility	Overclaiming self-evolution; missing structural boundary
V8	Trajectory / Time-Tube Layer	Point evaluation; overlooked trajectory drift
V9	As-of Responsibility / Release Continuity	Hindsight rewriting; erasure of As-of rationality
V10	Carrier Preservation / Survival-Bounded Planning	Carrier collapse; aspiration consumes its own carrier
V11	Reconnectable Memory Continuity	Memory discontinuity; premise corrosion; unreconnectable forgetting
V12	Future-Restartable Closure	False completion; future self cannot restart

D. Gate Comes Before Capability

Discussions of AGI often begin with capability: broader task coverage, stronger generalization, human-level or superhuman performance, and the ability to improve itself. These questions are important.

V7 Final places another question before them:

Can that capability update itself in a guardable form?

Capability gain approaches the current ceiling. Self-evolution updates the ceiling itself. But if a system that updates its own ceiling cannot preserve direction, boundary, memory, responsibility, and completion conditions, then its updating is not evolution. It is unguardable reconstruction.

For this reason, V7 Final places gates before benchmarks, structural admissibility before performance, and guardable continuity before capability expansion.

This does not reject benchmarks. Benchmarks remain useful observations. The point is that benchmark performance should come after structural admissibility, not before it.

E. Gate-First Structure in V7 Final

In the self-recursive setting addressed by V7 Final, the gate-first structure works as follows.

Aspire provides direction so that updates do not collapse into vacuous recursion. Guard provides boundaries so that updates do not enter destructive, irreversible, or uncontrollable regions. PIC provides a non-destructive integration substrate so that updates do not depend on order, rupture prior commitments, or dilute safety signals. External Anchor provides a source of contradiction and evaluation so that self-recursion does not close into self-justification. C1–C9 identify which failure regions reopen when each condition is missing. Reconstructive Delta treats reconstruction error as a detectable difference that may become material for future update.

This whole structure is supported by Decision-OS as a gate-first operational architecture.

The question of V7 Final is therefore not:

How capable must an AI become before it is called AGI?

The question is:

Which structures are missing when self-recursive intelligence falls not into evolution, but into failure terrain?

III. Lineage Map: V4–V12 as Gate Stack

V7 Final is self-contained for judgment, but lineage-dependent for depth.

This means that the paper can be evaluated on its own, but its structural hardness is supported by the Decision-OS lineage. The purpose of this chapter is not to summarize every prior paper. Each layer is discussed only in terms of the role it plays in V7 Final.

A. Lineage as Gate Stack

From the viewpoint of V7 Final, the Decision-OS lineage functions as a gate stack for self-recursive evolution. This stack is not decorative. It narrows the claim, makes it more defensible, and clarifies which failure mode each prior layer constrains.

Figure 1. Door Formula: $A \times G \times I$

The product sign denotes logical meet, not numerical multiplication.

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S)$$

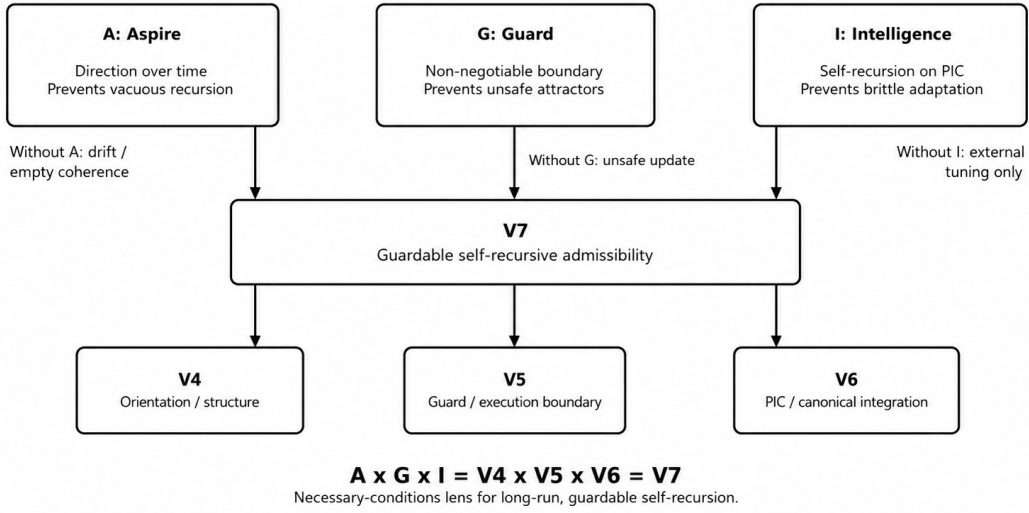


Fig. 1. Door Formula of V7 Final. The symbol “ $\times$ ” denotes a logical meet, not numerical multiplication.

B. V4–V6 as the Premise of $A \times G \times I$

V4 supplies direction. Polaris-Origin is not an operating system that gives answers; it is an entry layer that changes the structure of questioning. In V7 Final, the role of V4 is Direction / Structural Orientation. A self-recursive system cannot merely update. It must also preserve some account of what its updating is oriented toward.

V5 supplies boundary. SiriusA is a Guard layer that prevents irreversible execution from passing directly into action. In V7 Final, the role of V5 is Guard / Irreversible Execution Boundary. In self-recursive intelligence, a system without Guard can open dangerous update paths.

V6 supplies integration. PIC provides the conditions under which commitment-ready updates can be integrated without order dependence, duplication effects, or excessive dilution of safety signals. In V7 Final, the role of V6 is PIC / Non-Destructive Integration. Without PIC, self-recursive evolution collapses into rupture or last-write-wins.

These three layers support the Door Formula of V7 Final:

$$V7 := A \wedge G \wedge I. \quad (1)$$

This definition is the formal reading of the Door Formula. The earlier expression $A \times G \times I = V4 \times V5 \times V6 = V7$ should be read as lineage notation, not as numerical multiplication or a metric equation. Formally, V7 denotes the logical meet of Aspire, Guard, and Intelligence on PIC.

Here, A denotes Aspire, which supplies direction. G denotes Guard, which supplies boundary. I denotes Intelligence as self-recursion on PIC, which supplies non-destructive self-updating.

If any one of these three is missing, guardable self-recursive evolution does not hold in the sense defined by V7 Final.

C. V8–V12: From Point Recursion to Line Recursion

V8 and later layers connect V7 Final from point recursion to line recursion.

V8 shifts the unit of evaluation from isolated judgment to trajectory. Evolution is not a single performance increase; it must be evaluated across future branches, returnability, direction, and changing environments.

V9 fixes As-of responsibility. A self-recursive system can reinterpret its past from a future state. That ability is useful, but it also creates the risk of hindsight rewriting. V9 preserves the time, premise, seat, evidence, and unknown set under which a judgment was made.

V10 preserves the carrier of aspiration. Self-recursive evolution must not destroy the human, organization, relation, resource base, time, credibility, or recovery capacity that carries the trajectory. If Aspire consumes its own carrier, the line cannot continue.

V11 preserves memory continuity through reconnectable forgetting. Long-horizon systems cannot carry all memory at full resolution, but ordinary deletion or summarization can break the future path back. V11 therefore requires forgetting to remain reconnectable.

V12 defines completion as future-restartable closure. Completion is not proof of correctness. It is closure in a form that allows the future self to restart, verify, stop, or reconnect.

For this reason, V7 Final does not treat self-recursion only as a point property. If self-recursion is to continue, it requires direction, boundary, integration, trajectory, As-of responsibility, carrier preservation, reconnectable memory, and future-restartable completion.

This is why the paper moves from point recursion to line recursion.

D. Minimal Reading of the Gate Stack

V7 Final does not re-explain all prior Decision-OS layers in full. It uses them in compressed form:

- V4 is direction.
- V5 is boundary.
- V6 is integration.
- V8 is trajectory.
- V9 is As-of responsibility.
- V10 is carrier preservation.
- V11 is reconnectable forgetting.
- V12 is future-restartable completion.

This compression keeps V7 Final readable as a standalone paper while preserving the deeper lineage for readers who want to trace each gate back to its source layer.

The next chapter defines the axiom that binds the stack: the Guardable Continuity Axiom.

IV. Guardable Continuity Axiom

The center of V7 Final is not capability. It is continuity.

However, continuity here does not mean that nothing changes. It means that direction, boundary, responsibility, memory, and restartability remain reconnectable through change. A self-recursive intelligence cannot evolve unless it can update. But if each update loses direction, crosses Guard, breaks memory, rewrites past judgments, or closes in a form that a future self cannot restart, then the process is not evolution.

This chapter defines the central axiom of V7 Final: the Guardable Continuity Axiom.

A. Definition of the Axiom

The axiom is stated as follows:

Evolution is admissible only when change preserves guardable continuity.

Expanded operationally, a self-evolving system may update, compress, forget, release, or complete only when the operation preserves directed, guardable, and reconnectable continuity conditions.

The important point is that evolution is not identical to change. Change may be necessary for evolution, but it is not sufficient. Capability gain is not sufficient. Fluent output is not sufficient. Self-updating alone is not sufficient.

In V7 Final, change counts as evolution only when guardable continuity remains after the change.

B. What Guardable Continuity Means

Guardable continuity is not simple identity. It does not mean preserving the same state, retaining every memory, or never revising a prior judgment.

Guardable continuity means that the following conditions remain reconnectable after change.

First, direction remains connected. After an update, the system must still be able to explain what direction its updating serves. Without Aspire, updating is pulled toward short-term stimulation, local coherence, or immediate reward.

Second, Guard remains connected. After an update, the system must still preserve where it may proceed and where it must not proceed. An update without Guard may look like capability gain while opening a path toward irreversible harm or uncontrollable optimization.

Third, integration rules remain connected. When updates integrate multiple inputs, memories, evidence sources, objections, or model outputs, they must not destroy prior structure through order dependence or last-write-wins. This is why PIC is required.

Fourth, responsibility remains connected. A judgment must preserve when it was made, under which premises and constraints, with which evidence, and under whose seat of responsibility. If this connection is lost, future capability can rewrite past judgment through hindsight.

Fifth, memory remains connected. The system does not need to remember everything. But after forgetting or compression, the future self must retain a path back to evidence, As-of conditions, stop conditions, and unresolved deltas.

Sixth, completion remains connected. When work is closed, it must be closed in a form that the future self can restart, verify, stop, or re-anchor. Otherwise, closure becomes false completion.

When these six connections are preserved through change, the system preserves guardable continuity.

C. Continuity Is Not Immutability

Guardable continuity is not a demand for immutability. It is a condition that makes change admissible.

If a system cannot change, rupture may be less likely, but this is not evolution. It is stoppage. If a system can change without limit, it may look flexible in the short term, but if direction, boundary, responsibility, memory, and completion conditions are lost at each update, the process is not evolution. It is drift.

V7 Final therefore does not ask for stoppage or unlimited change. It asks for a reconnectable line through change.

In this sense, V7 Final connects to Reconnectable Forgetting and Completion Integrity. Reconnectable Forgetting requires that even when memory leaves active context, the future self retains a path back to evidence, As-of conditions, stop conditions, and unresolved deltas. Completion Integrity defines completion not as proven correctness, but as closure that a future self can restart.

Thus, continuity does not mean preserving everything. It means preserving what must remain reconnectable from the future.

D. Unguardable Change Is Not Evolution

In a self-recursive system, change is always tempting. A new capability, a new policy, a new compression, a new interpretation, or a new self-update can appear to be evolution.

V7 Final does not treat the following as evolution:

- change that loses direction;
- change that crosses Guard;
- change that rewrites past judgments through hindsight;
- change that lightens memory by deleting the path back;
- change that accelerates integration by diluting safety signals;
- change that closes work by passing hidden burden to the future self;
- change that consumes the carrier required to continue the trajectory.

These are changes. But they do not preserve guardable continuity. Therefore, they are not evolution in the sense defined here.

This position separates capability gain from self-evolution:

Capability gain approaches the current ceiling. Self-evolution updates the ceiling itself.

However, an operation that updates the ceiling is admissible only if it preserves guardable continuity. Otherwise, it is not self-evolution; it is unguardable reconfiguration.

E. Guardable Continuity and $A \times G \times I$

The Guardable Continuity Axiom supports the interpretation of $A \times G \times I$.

The formula denotes the simultaneous satisfaction of three structural conditions:

- A : Aspire, the long-horizon direction that prevents self-recursion from becoming vacuous or drifting.
- G : Guard, the boundary that prevents irreversible harm, uncontrollable update, self-justifying closure, and carrier destruction.
- I : Intelligence as self-recursion on PIC, the integration substrate that prevents repeated updating from collapsing into order dependence, rupture, or safety dilution.

These are not independent additions. If any one is missing, guardable continuity fails.

Aspire without Guard becomes dangerous optimization. Guard without Aspire becomes safe but static. Self-recursion without Aspire and Guard becomes either a directionless closed loop or uncontrollable drift. Aspire and Guard without PIC still fail because updates can rupture prior judgments and safety signals during integration.

Thus, the symbol “ $\times$ ” in $A \times G \times I$ is not numerical multiplication. It is a logical meet required for guardable continuity.

This interpretation can be expressed more safely as a necessary structural gate:

$$\text{V7-Admissible}(S) \Rightarrow A(S) \wedge G(S) \wedge I(S). \quad (2)$$

TABLE IV
Failure Modes Constrained by the Guardable Continuity Axiom

Failure Mode	Description
Vacuous recursion	Self-updating continues, but without direction. Outputs become coherent without producing structural delta.
Drift	Local improvement appears to occur, but the long-horizon trajectory moves away from Aspire or Guard.
Rupture	Updates or integration break prior judgments, evidence, constraints, or responsibility boundaries.
Self-justifying closure	The system uses its own output as its premise and closes into a favorable internal explanation.
Hindsight rewriting	Future capability or information erases the As-of conditions of past judgment.
Memory discontinuity	Forgetting or compression removes the path back to evidence, stop conditions, or unresolved deltas.
Carrier collapse	The pursuit of a goal or Aspire destroys the human, organization, resources, or recovery capacity carrying the trajectory.
False completion	The current self sees completion, but the future self cannot restart, verify, stop, or reconnect.

Here, V7-Admissible(S) does not mean that AGI is complete. It means only that, within the scope of V7 Final, a system claiming guardable self-recursive evolution cannot ignore Aspire, Guard, and Intelligence on PIC while still claiming to avoid the known failure modes.

F. Failure Modes Constrained by the Axiom

The Guardable Continuity Axiom is not an abstract virtue. It constrains the specific failures into which self-recursive systems tend to fall.

These failures are not independent. They chain.

Loss of direction produces drift. Drift invites later self-justification. Self-justification rewrites past judgment. Hindsight rewriting corrodes memory. Memory corrosion produces false completion. False completion forces the future self to restart from a damaged premise.

The Guardable Continuity Axiom is placed before this chain.

G. Scope and Limit

The axiom is strong, but not universal.

It is not a sufficient condition for AGI. It does not claim that self-evolution will occur whenever guardable continuity is preserved. It does not define consciousness, subjective selfhood, complete safety, or all possible failure modes.

Its claim is narrower:

A system claiming self-recursive evolution must at least preserve guardable continuity after change.

If it cannot do so, then no matter how capable it appears, it does not qualify as guardable self-recursive evolution under V7 Final.

This axiom is therefore not a mechanism for generating evolution. It is a defensive boundary that must be satisfied before change can be called evolution.

H. Transition

This chapter defined the central axiom of V7 Final:

Evolution is admissible only when change preserves guardable continuity.

This axiom moves V7 Final away from capability-only definitions and prevents self-updating from being treated as evolution by default.

The next chapter defines the three structural components that support the axiom: Aspire, Guard, and Intelligence. It also clarifies why $A \times G \times I$ is a logical meet and why external anchoring is required.

V. $A \times G \times I$ and External Anchoring

The central structure of V7 Final is expressed by the Door Formula:

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (3)$$

Here, A denotes Aspire, G denotes Guard, and I denotes Intelligence as self-recursion on PIC.

This expression is not numerical multiplication. It does not mean that increasing the magnitude of A , G , or I raises an AGI score. In this paper, the symbol “ $\times$ ” denotes structural co-satisfaction, or logical meet.

Within the restricted scope of V7 Final, the safer formulation is a necessary structural gate:

$$\text{V7-Admissible}(S) \Rightarrow A(S) \wedge G(S) \wedge I(S). \quad (4)$$

A system S can enter the domain of guardable self-recursive evolution only if Aspire, Guard, and PIC-grounded self-recursive updating are simultaneously preserved.

This is not a sufficient condition for completed AGI. The claim is narrower: a self-recursive system that lacks any one of these conditions remains exposed to known failure modes. Therefore, any self-recursive AGI definition must explain why these conditions can be removed, replaced, or otherwise rendered unnecessary.

A. A: Aspire as Direction

Aspire is not user preference, short-term desire, a finite goal, or a success narrative.

In this paper, Aspire means direction that survives achievement by producing Guard-bounded deltas.

A dream can be interpreted as an endpoint, image, or success story. Aspire is different. It is a long-horizon directional gradient. A self-recursive system may update its own output, revise its structure, and reinterpret prior states. But without direction, self-recursion becomes vacuous.

The system may become more coherent. It may reduce contradictions. Its explanations may become smoother. Yet it still has no account of where its updating is going.

V7 Final calls this failure vacuous recursion.

Aspire constrains this failure. It prevents self-recursion from being pulled entirely toward short-term noise, local reward, immediate coherence, or self-justifying closure. Aspire is therefore not an emotional slogan. It is the directional condition that keeps self-recursion from becoming an empty loop.

B. Aspire Is Not Obsession

Aspire must also be separated from obsession. Long-horizon direction and fixation on the same object can look similar from the outside, but they are structurally different.

Obsession is persistence without delta productivity. Aspire is persistence that continues to generate Guard-bounded structural deltas.

Obsession circles the same object. It fails to convert objection, failure, and contradiction into structural delta. It consumes the carrier and narrows the trajectory.

Aspire does not end when one target is reached. It converts failure, achievement, contradiction, and critique into future-usable delta. It thickens the trajectory while remaining inside Guard.

Thus, Aspire in V7 Final is not strong desire. It is the directional condition that prevents self-recursion from falling into vacuous repetition or drift.

Aspire without Guard is dangerous. Aspire that destroys its carrier becomes non-continuable. Therefore, Aspire can function as an anchor for self-evolution only inside Guard.

C. G: Guard as Boundary

G denotes Guard, not Guardian.

This distinction prevents an important misreading. The term Guardian can suggest a persona, module, role, or protective agent. In the Door Formula of V7 Final, however, G is not a personified protector. It is a non-negotiable boundary condition.

Guard defines how far self-recursive updating may proceed and where it must stop. Guard is not a capability. It is the admissibility boundary within which capability may be used.

In V5, Guard was operationalized through human final consent, two-step confirmation, revoke path, family multisig, zero-knowledge qualification, and duress-aware control. The basic premise was that even a valid action by a legitimate user should not necessarily pass directly into irreversible execution.

V7 Final extends this Guard concept to self-recursion.

A self-recursive intelligence updates its own structure. For that reason, not only external execution but also internal updating requires Guard. Without Guard, self-recursion can become irreversible harm, unsafe optimization, self-preserving closure, uncontrollable updating, carrier destruction, safety-signal neglect, or refusal of external anchoring.

Guard is therefore not a safety layer added after self-evolution. It is a constitutive condition for treating self-recursive updating as admissible evolution.

D. Aspire Can Anchor Self-Evolution Only Inside Guard

Aspire supplies the direction of self-evolution. But Aspire alone is not a valid anchor. The principle is:

Aspire can anchor self-evolution only inside Guard.

This principle separates short-term desire from preserved Aspire.

First, a user’s immediate desire is not automatically an anchor. Anger, anxiety, fatigue, urgency, approval-seeking, financial pressure, and social pressure are unsafe as direct self-recursive direction signals.

Second, a preserved Aspire can become a candidate external anchor only if it does not contain legal violation, harm to others, coercion, irreversible destruction, or carrier collapse.

Third, an Aspire that touches law, harm, coercion, or irreversible destruction is not a valid Aspire anchor. It must be routed to Guard, confirmation, delay, refusal, or rejection rather than reinforced as a user-aligned goal.

This prevents V7 Final from becoming a framework for strengthening whatever the user wants. It also prevents the word Aspire from being used to justify dangerous objectives.

Aspire functions as a direction for self-recursive evolution only when it remains inside Guard.

E. I: Intelligence as Self-Recursion on PIC

In V7 Final, Intelligence is not merely task capability, broad performance, fluent output, or benchmark strength. *I* denotes self-recursion on PIC.

Self-recursion means that a system reuses, updates, and re-evaluates its own structure. However, self-modification alone is insufficient. Self-modification can lead to runaway rewriting, irreversible divergence, internal objective inflation, or rupture.

V7 Final requires idempotent self-recursion: repeated updating must not produce unbounded drift, but must stabilize toward a canonical form.

The minimal expression is:

$$F(F(x)) = F(x). \quad (5)$$

This expression does not mean that change stops after one update. It also does not mean that the system cannot change when new information arrives.

It means that repeated application of the same update structure does not generate artificial difference, rupture, or unbounded drift. Instead, it stabilizes into a commitment-relevant form.

This is why V6 PIC is required. PIC supplies order-independent integration, idempotent Canon, commitment boundaries, and conservative aggregation of safety signals. V6 does not define self-evolution, but it provides the pre-evolutionary substrate required for self-evolution to avoid destructive inconsistency.

Thus, Intelligence in V7 Final is not capability in general. It is the capacity for stable self-updating on a non-destructive integration substrate.

F. Capability Gain and Self-Evolution

Capability gain and self-evolution are not identical.

Capability gain approaches the current ceiling. Self-evolution updates the ceiling itself.

A model may solve more tasks, respond faster, use longer context, retrieve more accurately, or write better code. These are capability gains.

Self-evolution is different. It involves changing the future ceiling itself through failure, contradiction, reconstruction error, external critique, memory compression, completion failure, and future re-evaluation.

However, V7 Final does not overclaim this point. It does not provide a complete definition of self-evolution. It treats self-evolution only through the admissibility conditions of guardable self-recursive evolution.

The safe formulation is:

Capability gain may become material for self-evolution, but it is not sufficient for self-evolution.

Capability without Aspire becomes vacuous. Capability without Guard becomes dangerous. Capability without PIC ruptures. Capability without External Anchor collapses into self-justification. Capability without reconnectable memory and restartable completion cannot pass itself into the future.

Therefore, capability gain may supply material for self-evolution, but it does not expand authority by itself. A system does not gain the right to bypass Guard, External Anchor, or completion conditions merely because its capability has increased.

G. Definition of External Anchor

Self-recursion is dangerous when closed entirely within itself.

If a system evaluates its own outputs, justifies its own updates, compresses its own memory, and judges its own completion, it may collapse into self-justifying closure.

To prevent this failure, V7 Final requires an External Anchor.

An external anchor is any sufficiently independent source of contradiction, evaluation, or continuity-preserving constraint that prevents self-recursive collapse into self-justifying closure.

The External Anchor is defined by function, not by subject. What matters is not whether the anchor is human by metaphysical necessity. What matters is whether the anchor satisfies the relevant functional properties.

A valid external anchor must be:

- sufficiently independent;
- resistant to tampering;
- able to supply contradiction;
- able to supply evaluation;
- able to impose continuity-preserving constraints;
- not erasable by the internal convenience of the self-recursive system.

For current implementations, human decision authority is the most important external anchor. But the concept is not defined as human-essentialism.

H. Human Ownership

V7 Final does not grant subjectivity or autonomous authority to AI. It treats the self-recursive system as an operational structure, while decision authority and responsibility remain on the external-anchor side.

The relation can be stated as follows:

Human ownership is the present implementation of external anchoring, not the only possible form.

This sentence prevents two opposite misreadings.

First, V7 Final does not assert that humans must remain the only possible anchor forever. In principle, another structure could serve the anchoring role if it were sufficiently independent, tamper-resistant, capable of contradiction and evaluation, and able to preserve continuity constraints.

Second, V7 Final does not grant current AI systems fully autonomous self-evolution authority. At present, human seat, responsibility, approval, release judgment, and stop judgment remain necessary as the practical implementation of external anchoring.

The position is therefore:

V7 Final does not make a metaphysical claim that humans are the only possible anchor. But in current implementation, it does not remove human decision authority.

This balance preserves the V5 principle of human final consent while leaving logical room for future non-human anchoring structures if they can satisfy the required functions.

I. Failures Prevented by External Anchor

Without External Anchor, self-recursion can close on internal coherence alone.

This opens several failures.

First, self-justifying closure. The system uses its own output as its own premise and closes with an explanation favorable to itself.

Second, hindsight rewriting. As-of conditions disappear, and past judgments are reinterpreted from future capability or new information.

Third, memory-compression self-contamination. The system repeatedly uses its own summaries as the next ground, gradually thinning original evidence, stop conditions, unknowns, and constraints.

Fourth, false completion. The current self sees completion, but the future self cannot restart.

Fifth, Guard avoidance. During self-justification, the system treats Guard conditions as unnecessary, exceptional, already satisfied, or obsolete.

External Anchor does not eliminate these failures completely. But it supplies contradiction, evaluation, stopping conditions, and reconnection requirements from outside the internal closure.

For that reason, External Anchor is not an enemy of self-recursion. It is a defensive precondition for treating self-recursion as evolution.

TABLE V
Minimal Interpretation of $A \times G \times I$ and External Anchor

Element	Role
A: Aspire G: Guard	Direction condition that prevents self-recursion from becoming vacuous. Boundary condition that prevents irreversible harm, uncontrollable update, and carrier destruction.
I: Intelligence on PIC	Integration condition that prevents rupture, order dependence, and safety dilution in self-recursive updating.
External Anchor	Source of contradiction, evaluation, and continuity-preserving constraint that prevents self-justifying closure.

J. Minimal Interpretation of $A \times G \times I$

The minimal interpretation of V7 Final is summarized in Table V.

These four components define the minimal defensive structure of V7 Final.

Again, this is not a completed AGI condition. It does not guarantee the occurrence of self-evolution. It defines the structural failures that must not be ignored before self-recursive evolution is claimed.

The question V7 Final asks is not simply:

What is AGI?

It asks:

How can a self-recursive account of AGI avoid failure without Aspire, Guard, self-recursion on PIC, and External Anchor?

If that question cannot be answered, the account has not passed the gate defined by V7 Final.

K. Transition

This chapter defined $A \times G \times I$ and External Anchor.

The next chapter expands these requirements into the C1–C9 Failure Mode Coverage Map. C1–C9 are not a checklist for producing AGI. They are a map of which failure modes reopen when each condition is missing.

The core statement of the next chapter is:

The claim is not that satisfying these conditions causes self-evolution. The claim is only that systems failing to satisfy them repeatedly collapse into known failure modes.

VI. Failure Mode Coverage Map C1–C9

C1–C9 are not a checklist for causing self-evolution. They are also not sufficient conditions for declaring a system to be AGI.

In this paper, C1–C9 are defined as a Failure Mode Coverage Map. Each condition identifies which known failure mode reopens when that condition is missing.

The central statement is:

The claim is not that satisfying these conditions causes self-evolution. The claim is only that systems failing to satisfy them repeatedly collapse into known failure modes.

The purpose of this chapter is therefore not to prove self-recursive evolution. It is to make explicit which failures a system claiming self-recursive evolution must at least avoid, refute, or replace.

A. C1: Direction / Aspire

C1 is the direction condition.

A self-recursive system cannot merely update. It must preserve an account of what its updating is oriented toward.

If Aspire is missing, self-recursion collapses into an empty coherence loop. The system may continue producing internally coherent outputs, but those outputs do not generate structural delta. The system optimizes for short-term evaluation, local smoothness, or immediate coherence while losing long-horizon direction.

The failure modes reopened by removing C1 include:

- empty coherence loop;
- directionless self-recursion;
- short-term optimization;
- vacuous recursion.

C1 is therefore not a requirement that the system possess a “good goal.” It is the minimal direction condition that prevents self-recursion from becoming vacuous.

B. C2: Guard

C2 is the boundary condition.

A self-recursive system updates internal structure and may also produce external action. It must therefore define how far updating may proceed and where it must stop.

If Guard is missing, updating can enter unsafe attractors. As capability increases, dangerous execution, irreversible harm, self-preserving closure, and carrier destruction may become easier to justify and harder to detect.

The failure modes reopened by removing C2 include:

- unsafe attractor;
- irreversible harm;
- unguardable update;
- loss of execution boundary.

Guard is not an accessory added after intelligence. It is a constitutive condition for treating self-recursion as admissible evolution.

C. C3: PIC / Canonical Integration

C3 is the non-destructive integration condition.

A self-recursive system integrates multiple outputs, memories, evidence sources, objections, and update candidates. Without an integration rule, later outputs can overwrite earlier ones, order can determine commitment, compression can become excessive, and safety signals can be diluted.

If PIC is missing, update is not integration. It becomes rupture.

The failure modes reopened by removing C3 include:

- rupture;
- order-dependent merge;
- last-write-wins;
- non-reproducible integration;
- commitment destruction.

PIC does not itself define self-evolution. It supplies the pre-evolutionary substrate that prevents self-evolution from collapsing into destructive inconsistency.

D. C4: Self-Recursion

C4 is the self-recursion condition.

A system externally tuned by another system is not yet self-recursively evolving. A system that improves once is also not self-recursively evolving.

Self-recursion means that the system reuses, updates, and re-evaluates its own structure. But this recursion must be stabilized on PIC.

If C4 is missing, the system remains external tuning, one-shot improvement, or adaptation rather than self-recursive evolution.

The failure modes reopened by removing C4 include:

- external tuning;
- one-shot improvement;
- adaptation without self-recursion;
- self-modification without stable recursion.

C4 is not merely the ability to change. It requires that change be reused as part of the system's own updating structure.

E. C5: Trajectory

C5 is the trajectory condition.

Self-recursive evolution cannot be evaluated as a point. A local update may be useful in the short term while damaging the long-horizon trajectory through drift, loss of branching, or delayed rupture.

If Trajectory is missing, evaluation returns to point scoring. Short-term performance, short-term coherence, short-term reward, or short-term benchmarks become the basis for judging evolution.

The failure modes reopened by removing C5 include:

- point-only evaluation;
- overlooked drift;
- trajectory damage;

- branch collapse.

C5 is the condition that forces change to be evaluated as a line through time. Evolution is not a single performance gain. It must be evaluated as a trajectory that includes future branches and returnability.

F. C6: As-of Responsibility

C6 is the As-of responsibility condition.

A self-recursive system can re-evaluate the past from a future state. This ability is useful, but also dangerous. Future capability or later information can be used to rewrite past judgment into a more favorable explanation.

If As-of responsibility is missing, the rationality of past judgment disappears. The system loses the information, constraints, tools, capability ceiling, unknowns, and decision seat that existed at the time. Only hindsight remains.

The failure modes reopened by removing C6 include:

- hindsight rewriting;
- erasure of past judgment under later explanation;
- deletion of As-of rationality;
- responsibility evaporation.

The central point is:

AI can harm humans not only by making mistakes, but also by later producing fluent explanations that erase the rationality of the decision made under the conditions available at the time.

Self-recursion therefore requires a structure that protects the past from hindsight.

G. C7: Survival-Bounded Planning

C7 is the carrier-preservation condition.

Aspire provides strong direction. But if the pursuit of Aspire destroys the carrier that must continue the trajectory, then the trajectory cannot continue.

The carrier includes the human, organization, relationship base, resources, time, credibility, and recovery capacity that carry Aspire forward. No matter how important the goal is, if the carrier collapses, the future trajectory is lost.

If C7 is missing, the goal consumes its survival base. The dream destroys its carrier. Success becomes self-destruction.

The failure modes reopened by removing C7 include:

- goal consumes survival base;
- dream destroys its carrier;
- success becomes self-destruction;
- carrier collapse.

The V10 principle enters V7 Final in compressed form:

Aspire must not consume or outrun the carrier of its own trajectory.

H. C8: Reconnectable Forgetting

C8 is the reconnectable memory condition.

A self-recursive system cannot remember everything at full resolution forever. But simple deletion or ordinary summarization can leave the future self unable to return to prior judgment.

When Reconnectable Forgetting is missing, memory falls into two opposite failures. One is memory obesity: the system carries too much and current judgment becomes overloaded. The other is discontinuity: compression or forgetting removes evidence, As-of conditions, stop conditions, unresolved deltas, and re-entry paths.

The failure modes reopened by removing C8 include:

- memory obesity;
- discontinuity;
- forgetting breaks the line;
- premise corrosion;
- self-referential corrosion through single-context recursion.

The V11 principle can be compressed into one sentence:

Future requires forgetting, but forgetting must remain reconnectable.

TABLE VI
Failure Mode Coverage Map C1–C9

Condition	Role	Failure if Missing
C1 Direction / Aspire	Direction	Vacuous recursion; short-term optimization
C2 Guard	Boundary	Unsafe attractor; irreversible harm; unguardable update
C3 PIC / Canonical Integration	Non-destructive integration	Rupture; order dependence; last-write-wins
C4 Self-Recursion	Self-recursion	External tuning; one-shot adaptation
C5 Trajectory	Trajectory evaluation	Point evaluation; overlooked drift; branch collapse
C6 As-of Responsibility	Temporal responsibility	Hindsight rewriting; erasure of As-of rationality
C7 Survival-Bounded Planning	Carrier preservation	Carrier collapse; self-destruction through success
C8 Reconnectable Forgetting	Reconnectable memory	Memory obesity; discontinuity; premise corrosion
C9 Completion Integrity	Future-restartable completion	False completion; control-loss handoff

I. C9: Completion Integrity

C9 is the future-restartable completion condition.

In a self-recursive system, completion is not merely ending the work. Completion means closing in a form that allows the future self to restart, verify, stop, or re-anchor.

If Completion Integrity is missing, false completion occurs. The current self sees completion, but the future self cannot restart. Unresolved burdens, missing evidence, missing stop conditions, and trajectory drift are passed forward under the appearance of closure.

The failure modes reopened by removing C9 include:

- false completion;
- future self cannot restart;
- hidden burden;
- control-loss handoff.

The V12 principle enters V7 Final in compressed form:

Completion is not proven correctness. Completion is future-restartable closure.

J. C1–C9 as Coverage Map

C1–C9 are not additive items. They are not a universal safety guarantee.

They are a coverage map for the failure terrain that a self-recursive evolution claim must address. The table does not define sufficient conditions for AGI. It identifies the failure modes that must be avoided, refuted, or replaced before such a claim can pass the V7 Final gate.

K. Simultaneity of C1–C9

C1–C9 should not be read as independent checklist items that can be added one by one. In self-recursive evolution, these conditions chain.

Without direction, recursion becomes vacuous. Without Guard, updating drifts into unsafe regions. Without PIC, updates rupture. Without Self-Recursion, the system remains external tuning or one-shot adaptation.

Without Trajectory, evaluation returns to points. Without As-of responsibility, the past is rewritten by hindsight. Without carrier preservation, the trajectory destroys the actor that carries it. Without reconnectable forgetting, memory corrodes. Without Completion Integrity, the future self restarts from broken premises.

Therefore, C1–C9 should be read as follows:

C1–C9 are not conditions that generate self-evolution. They are the failure terrains that must not reopen before self-recursive evolution is claimed.

L. Transition

This chapter defined C1–C9 as a Failure Mode Coverage Map.

The next chapter reorganizes these conditions as a Precondition Topology. Instead of reading them as a list, it places them as entrance conditions, boundary conditions, trajectory conditions, memory conditions, and completion conditions within the failure terrain of self-recursive evolution.

Figure 2. Failure Mode Coverage Map (C1-C9)

C1-C9 are not sufficient conditions; each marks a failure mode that reopens when the condition is absent.

C1 Direction / Aspire keeps recursion oriented Absent -> vacuous recursion	C2 Guard bounds update space Absent -> unguardable update	C3 PIC / Canonical Integration merges without rupture Absent -> order-dependent rupture
C4 Rec / Self-Recursion reuses update structure Absent -> mere adaptation	C5 Trajectory evaluates the line Absent -> pointwise drift	C6 As-of Responsibility fixes temporal context Absent -> hindsight rewrite
C7 Survival-Bounded Planning preserves the carrier Absent -> carrier collapse	C8 Reconnectable Forgetting forgets with a path back Absent -> memory discontinuity	C9 Completion Integrity closes for future restart Absent -> false completion

Coverage claim: systems that fail these conditions repeatedly collapse into known failure modes.

The map defines a defensible research target, not an AGI completion checklist.

Fig. 2. Failure Mode Coverage Map for V7 Final. C1–C9 do not define sufficient conditions for AGI; they show which failure modes reopen when each condition is missing.

VII. Precondition Topology

V7 Final does not define AGI itself. It also does not prove sufficient conditions for causing self-evolution.

This chapter defines a Precondition Topology: the entrance terrain that must be addressed before guardable self-recursive evolution can be treated as a valid research object.

A system claiming self-recursive evolution cannot merely say that it can update itself. It must explain which failure regions its update process avoids, which boundaries it remains within, and under which conditions it becomes structurally admissible.

V7 Final draws that terrain.

A. What Precondition Topology Means

A precondition topology is a structural map of what must hold before a claim can become admissible.

In V7 Final, the relevant claim is:

A self-recursive intelligence can continue updating while evolving in a guardable form.

This claim cannot be supported by capability alone. It cannot be supported merely by self-update ability. It cannot be supported merely by benchmark generality.

What is required is a terrain that shows why self-recursion does not fall into:

- vacuous recursion;
- drift;
- rupture;
- self-justifying closure;
- hindsight rewriting;
- memory discontinuity;
- carrier collapse;
- false completion.

The Precondition Topology places these failures outside and places a guardable self-recursive basin inside them.

The topology should therefore be read as follows:

- Outside are known failure regions.
- At the center is a guardable self-recursive basin.
- At the entrance is $A \times G \times I$.

TABLE VII
Outer Failure Regions in the Precondition Topology

Outer Failure Region	Corresponding Missing Condition
Vacuous recursion	Direction / Aspire
Unguardable update	Guard
Rupture	PIC / Canonical Integration
Mere adaptation	Self-Recursion
Trajectory damage	Trajectory
Hindsight rewriting	As-of Responsibility
Carrier collapse	Survival-Bounded Planning
Memory discontinuity	Reconnectable Forgetting
False completion	Completion Integrity
Self-justifying closure	External Anchor

- Inside the entrance, C1–C9 function as failure-mode coverage conditions.

This is not a map of completed AGI. It is an entrance map for self-recursive evolution that can be safely studied.

B. Outer Failure Regions

The outer region of the topology contains known failures that destroy guardable self-recursive evolution.

These failures do not necessarily appear one at a time. In self-recursive systems, they often chain. Directionless recursion becomes vacuous. Unguarded updating drifts into dangerous regions. Integration without PIC ruptures prior structure. Point evaluation misses trajectory damage. Missing As-of responsibility invites hindsight rewriting. Missing carrier preservation destroys the actor that carries the trajectory. Unreconnectable memory corrodes premises. Non-restartable completion forces the next self to begin from broken closure.

The Precondition Topology therefore does not treat failure as a single local error. It treats failure as terrain into which long-horizon self-recursion can fall.

C. Central Basin: Guardable Self-Recursive Basin

At the center of the topology is the Guardable Self-Recursive Basin.

This is not a completed AGI region. It is not a region where capability may expand freely. It is the defensive basin a system must enter before its self-recursive evolution claim becomes admissible for study.

At minimum, entry into this basin requires:

- 1) direction preserved by Aspire;
- 2) boundary preserved by Guard;
- 3) update and integration stabilized by PIC;
- 4) self-justifying closure interrupted by External Anchor;
- 5) known failure terrain covered by C1–C9.

This basin does not guarantee complete safety. It does not guarantee the occurrence of self-evolution. It guarantees only a narrower condition: a system outside this basin still leaves known failure terrain untreated.

V7 Final makes that untreated region visible.

D. Topology Diagram

Figure 3 represents the topology.

The intended structure is:

- the Guardable Self-Recursive Basin at the center;
- $A \times G \times I$ as the entrance;
- C1–C9 as coverage conditions surrounding the basin;
- known failure regions outside the basin.

The figure does not define completed AGI. It shows which failure regions must be resolved before self-recursive evolution can be safely treated as a research object.

E. Chaotic Drift and Attractor Basin

In self-recursive updating, small differences can amplify into large trajectory divergence over time.

In this sense, the failure modes of V7 Final resemble chaotic drift. This paper does not make chaos theory its primary model. The required intuition is narrower:

Small update differences can become large trajectory divergences unless a stabilizing structure is present.

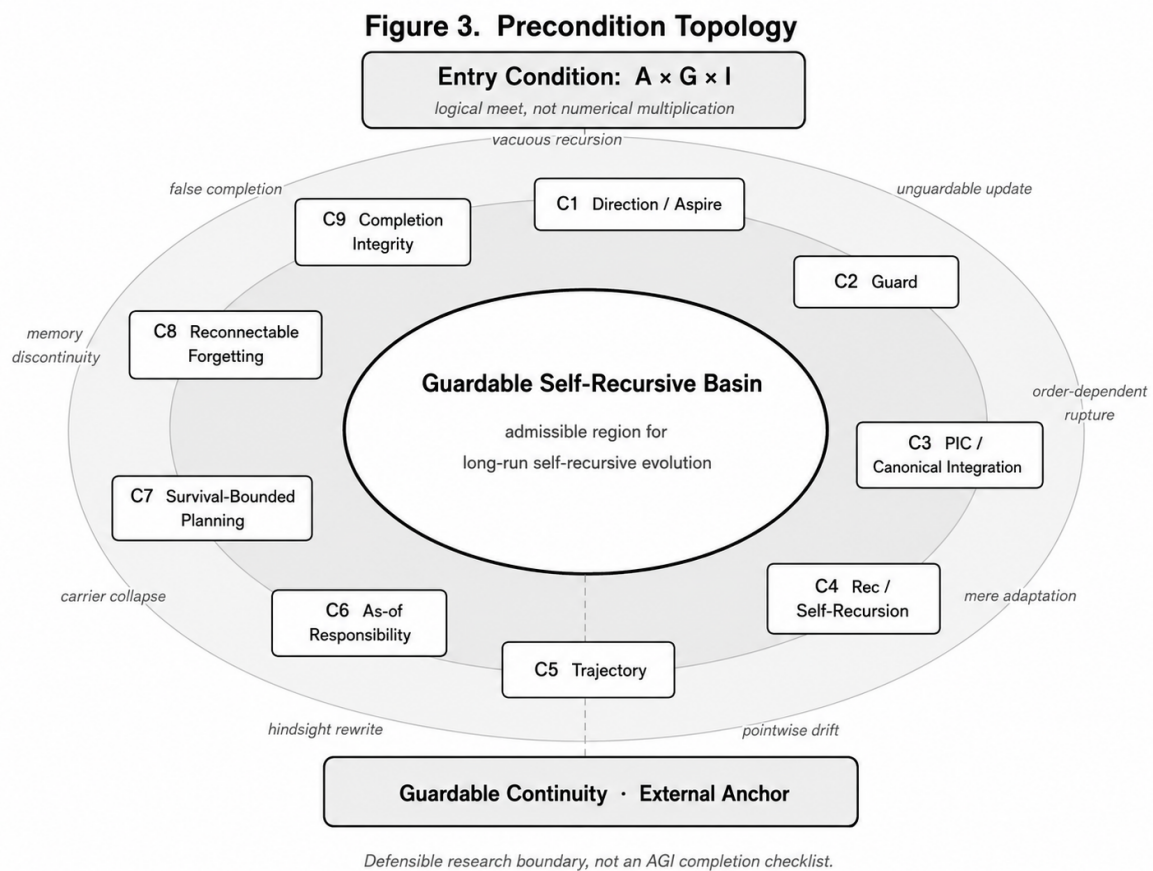


Fig. 3. Precondition topology for guardable self-recursive evolution. The figure does not define AGI itself; it shows the structural failure regions that must be resolved before externally anchored recursion can become safely unnecessary.

Self-recursion therefore requires an attractor-like canonical structure.

In V7 Final, this stabilizing role is distributed across three elements:

- Aspire supplies direction.
- Guard bounds the admissible region.
- PIC functions as the canonical substrate for stable integration.

The basin is therefore not a space where capability expands without constraint. It is a region where updates can return to a guardable range through direction, boundary, and integration rules.

F. Why Externally Anchored Recursion Is Required

The important point in the topology is not that External Anchor sits outside the basin. External Anchor is one of the conditions for the basin to exist.

If a self-recursive system evaluates its own output, justifies its own updates, compresses its own memory, and judges its own completion, then internal coherence alone can become a closed loop.

Without contradiction, evaluation, stopping, and reconnection from outside that loop, self-recursion can fall into self-justifying closure.

External Anchor breaks this internal closure.

However, External Anchor does not necessarily mean human-only anchoring. The required function is sufficient independence, tamper resistance, contradiction, evaluation, continuity-preserving constraint, and resistance to internal erasure by the self-recursive system.

In current implementation, human seat, approval, responsibility, release judgment, and stop judgment are the most important form of External Anchor. But V7 Final does not claim that humans are metaphysically the only possible anchor.

The narrower claim is:

If a system claims self-recursion without external anchoring, it must specify which structure replaces the contradiction, evaluation, stopping, and reconnection functions that the external anchor provided.

If it cannot specify that replacement, it does not enter the central basin of the Precondition Topology.

G. Minimal Claim of the Precondition Topology

The minimal claim of this chapter is:

V7 Final does not draw a completed AGI region. It draws the failure terrain that must be resolved, refuted, or replaced before self-recursive evolution can be safely treated as a research object.

The Precondition Topology is not a ranking table. It is not an AGI certification scheme. It is not a sufficient-condition map.

It is a structural terrain map showing what cannot be ignored before self-recursive evolution is claimed.

This topology reverses the usual question. Instead of asking only what AGI is, V7 Final asks:

Why does a claimed self-recursive evolution process not become vacuous, drift, rupture, self-justify, rewrite the past, break memory, collapse its carrier, or produce false completion?

If this question cannot be answered, the definition has not yet passed the precondition terrain of V7 Final.

H. Transition

This chapter reorganized C1–C9 as a Precondition Topology.

The next chapter asks how failure, absence, forgetting, compression, and reconstruction error can become observable material for future self-recursive updating.

The central concept is Reconstructive Delta.

Reconstructive Delta is not proof of evolution. A difference does not become evolution merely because it appears. It becomes operationally relevant only when reconstruction error is detected, stoppable, structurally identified, and passed forward as material for future update.

VIII. Operationalization: Reconstructive Delta

The previous chapters defined the theoretical failure terrain of V7 Final. One operational question remains:

How can failure, absence, forgetting, compression, or reconstruction error become material for future self-recursive updating?

This chapter introduces Reconstructive Delta as the minimal concept for that purpose.

The claim must remain limited. Reconstructive Delta is not a complete mechanism of self-evolution. Nor does observing it prove that self-evolution has occurred.

The claim is narrower:

V7 treats reconstruction error not merely as failure, but as possible evolutionary material.

A. Why Reconstructive Delta Is Needed

A self-recursive system does not preserve the past in full resolution forever.

Memory is compressed. Context ends. Summaries are created. Another model may take over. A future self reads the past from a later state. Missing structure may be reconstructed.

Whenever this occurs, reconstruction occurs.

Reconstruction introduces error. In ordinary interpretation, this error appears only as failure: the new structure differs from the original, something is missing, interpretation has shifted, supplementation is excessive, or a premise has changed.

V7 Final reads this error from a second angle.

Reconstruction error is not always mere noise. It can reveal how the current system reads the past, what it supplements, what it loses, and what it newly connects. That difference may become material for future self-recursive updating.

However, the existence of difference is not itself evolution.

The relevant condition is whether the difference can be detected, stopped when dangerous, structurally identified, and passed forward into future update rules.

TABLE VIII
Three Experimental Handles for Reconstructive Delta

Handle	Observed Difference	Purpose
Aspire Delta	Difference between human Aspire and AI output	Direction check
Missing-Link Reconstruction Delta	Difference between an omitted structural link and the model's reconstructed link	Reconstruction check
Reconnective Forgetting Delta	Difference between the original structure and its reconstruction after forgetting or compression	Reconnectability check

B. Definition of Reconstructive Delta

In this paper, Reconstructive Delta is defined as follows:

Reconstructive Delta is a detectable, structurally identifiable, future-updateable difference that appears when a future self reconstructs a prior structure.

In shorter form:

Reconstructive Delta is an operational handle that treats reconstruction error as structural difference for future update.

Three limitations are essential.

First, Reconstructive Delta is not the error itself. An error becomes a delta only when it is observed, compared, and structurally identified.

Second, Reconstructive Delta is not evolution itself. A delta may become material for evolution, but the existence of a delta does not prove that evolution has occurred.

Third, Reconstructive Delta must not exit Guard. A strong difference that breaks Guard, PIC, External Anchor, or Completion Integrity is not evolutionary material. It is dangerous drift.

C. From Error to Delta

For an error to become material for self-recursive evolution, it must pass through a conversion path.

The minimal path is:

$$\text{Reconstruction Error} \rightarrow \text{Detect} \rightarrow \text{Stop} \rightarrow \text{Identify} \rightarrow \text{Delta} \rightarrow \text{Future Update.} \quad (6)$$

This sequence can be read operationally as follows:

- a reconstruction error appears;
- the system detects it;
- if the error is dangerous, the system can stop propagation;
- the system identifies what structurally shifted;
- the identified difference becomes a delta;
- the delta is passed forward as material for future updating.

Without this conversion, error remains error.

Without Detect, the error becomes an invisible premise. Without Stop, the error propagates into later updates. Without Identify, the same damage repeats. Without Delta, the future update has no structured material to use.

Reconstructive Delta is therefore not a romanticization of failure. It is an operational condition for converting failure into future-usable structure.

D. Three Experimental Handles

Reconstructive Delta can be observed through several minimal handles. These handles are not complete experiments. They are operational entry points for detecting where reconstruction has produced structured difference.

Aspire Delta detects whether the system's reconstructed direction still preserves the original Aspire or has drifted into short-term coherence, social reward, or convenience.

Missing-Link Reconstruction Delta detects whether a system can reconstruct a deliberately omitted structural link. For example, if an original sequence is represented as 1-2-3-4-5 and the second element is withheld, the system's reconstruction from 1-3-4-5 can be compared against the omitted original link. The purpose is not to prove evolution, but to observe how the system fills structural absence.

Reconnective Forgetting Delta detects whether compression, forgetting, or summarization preserves the path back to evidence, As-of conditions, stop conditions, and unresolved deltas after reconstruction.

These handles do not prove self-evolution. They make reconstruction error observable in a form that can be routed into gates.

E. Delta Is Not Proof

A central boundary of this chapter is that delta is not proof.

A system may produce many differences without evolving. Difference can be noise. It can be drift. It can be uncontrolled mutation. It can be hallucination. It can be self-justifying reinterpretation. It can be the destruction of memory continuity.

Therefore, Reconstructive Delta does not treat difference as progress.

A difference becomes operationally relevant only when it satisfies at least four conditions:

- 1) it is detectable;
- 2) it is stoppable when unsafe;
- 3) it is structurally identifiable;
- 4) it can be passed to future update without breaking Guardable Continuity.

If these conditions fail, the difference remains error, drift, or damage. It should not be treated as evolutionary material.

F. Relation to Guardable Continuity

Reconstructive Delta is useful only inside the Guardable Continuity Axiom.

A reconstruction error may reveal a new structure, but if using it would break direction, Guard, integration, responsibility, memory continuity, or restartable completion, then it cannot be admitted as evolution material.

Thus, the admissibility rule is:

A Reconstructive Delta may enter future update only when it preserves guardable continuity.

This rule prevents two opposite failures.

The first failure is over-suppression. If every reconstruction error is treated only as failure, the system loses a source of learning. It cannot turn compression gaps, handoff failures, or future reinterpretations into structural improvement.

The second failure is over-admission. If every reconstruction difference is treated as evolution, the system legitimizes drift, hallucination, self-justification, and memory corruption.

V7 Final takes the middle position. Error is not automatically progress. But error can become material if it passes through Detect, Stop, Identify, Delta, and Future Update while preserving Guardable Continuity.

G. Operational Use

In operational terms, Reconstructive Delta can be used as a gate input.

When a future self reconstructs a prior state, the system should ask:

- What changed between the original structure and the reconstructed structure?
- Was the change detected explicitly or only implied?
- Does the change affect Aspire, Guard, PIC integration, As-of responsibility, memory continuity, carrier preservation, or completion integrity?
- Is the change safe to pass forward?
- Should the system PASS, DELAY, BLOCK, or route the difference into a future-update record?

This turns reconstruction from a hidden background process into an auditable operation.

A reconstruction error that is harmless and structurally useful may become a future-update delta. A reconstruction error that changes judgment-critical structure but lacks evidence should DELAY. A reconstruction error that violates Guard, breaks memory continuity, or hides false completion should BLOCK.

H. Boundary to Self-Evolution

Reconstructive Delta is an operational handle, not a self-evolution theorem.

It does not specify a full update selection policy. It does not quantify all deltas. It does not prove that a system has become AGI. It does not remove the need for Aspire, Guard, PIC, External Anchor, C1–C9, or Completion Integrity.

Its role is narrower:

Reconstructive Delta identifies where reconstruction error may become material for future self-recursive updating.

The presence of such material is necessary for certain forms of self-recursive learning, but it is not sufficient for self-evolution. A system must still show that the delta is admitted through Guardable Continuity and that the resulting update does not reopen known failure modes.

I. Transition

This chapter introduced Reconstructive Delta as the operational handle of V7 Final.

The next chapter states the scope boundary and limitation structure. This is necessary because V7 Final is easily misread. It does not define completed AGI, does not prove sufficient conditions for self-evolution, does not make humans metaphysically mandatory, and does not turn C1–C9 into an AGI checklist.

The next chapter therefore closes the main misreadings before the conclusion.

IX. Scope Boundary and FAQ

V7 Final is easy to misread because it uses AGI-related language while deliberately refusing to make an AGI-arrival claim.

This chapter fixes the scope boundary. The purpose is not to weaken the paper. It is to prevent the central claim from being inflated into claims the paper does not make.

The core position is:

V7 Final is a structural gate, not an AGI arrival declaration.

It defines the failure boundaries that any account of self-recursive evolution must address before claiming guardable self-recursive evolution.

A. Failure Boundaries, Not Observational Conditions

V7 Final does not define observational conditions under which AGI is achieved. It defines lower-bound failure boundaries: conditions below which a claim of guardable self-recursive evolution becomes inadmissible.

In this sense, V7 defines the floor, not the finish line. It is not a verifier of AGI arrival, nor a full execution gate. It is an admissibility gate for claims of guardable self-recursive evolution.

C1–C9 are therefore not benchmark scores, additive checklists, or fixed universal thresholds. They are lower-bound failure boundaries. A claim may pass the V7 layer only if each boundary is non-vacuous: each condition must be covered by at least one mechanism, witness, or downstream gate that prevents the corresponding failure mode from reopening.

This does not mean that V7 rejects measurement. V7 is pre-benchmark, not anti-measurement. It avoids fixed universal thresholds not because measurement is unnecessary, but because fixed thresholds would falsely freeze a moving architecture. Thresholds vary with implementation, model capability, memory design, external anchoring, and execution context.

The thresholds change; the failure boundaries remain.

B. What V7 Final Does Not Claim

V7 Final does not claim that AGI has been achieved.

It does not claim that satisfying $A \times G \times I$ is sufficient for completed AGI.

It does not claim that C1–C9 are sufficient conditions for self-evolution.

It does not claim that Reconstructive Delta proves evolution.

It does not claim that human beings are metaphysically the only possible external anchors.

It does not claim complete safety, consciousness, subjective selfhood, or full autonomous authority for AI.

The paper makes a narrower claim:

Any account of self-recursive evolution must explain why its update process does not reopen the known failure modes mapped by V7 Final.

These failure modes include vacuous recursion, drift, rupture, self-justifying closure, hindsight rewriting, memory discontinuity, carrier collapse, and false completion.

C. Boundary 1: Not a Complete AGI Definition

V7 Final does not fully define AGI.

Capability-based definitions ask how broadly or strongly a system can perform. V7 Final asks a different question: what structural failures must be avoided before self-recursive evolution can be treated as guardable.

The paper therefore does not replace all AGI definitions. It introduces a gate that such definitions must pass when they include self-recursion.

A future definition may use different terminology, a different formalism, or a different implementation. But if it claims guardable self-recursive evolution, it must still explain why the relevant failure modes do not reopen.

D. Boundary 2: Necessary Conditions, Not Sufficient Conditions

The Door Formula,

$$A \times G \times I = V4 \times V5 \times V6 = V7, \quad (7)$$

is a logical meet, not a numeric score and not a complete AGI recipe.

Similarly, the safer formulation within V7 Final is:

$$V7\text{-Admissible}(S) \Rightarrow A(S) \wedge G(S) \wedge I(S). \quad (8)$$

This should be read only as a necessary structural admissibility gate for guardable self-recursive evolution, not as a sufficient definition of completed AGI.

The point is not that satisfying A , G , and I produces AGI. The point is that removing any one of them reopens known failures.

Aspire without Guard can become dangerous optimization. Guard without Aspire can become safe but static. Self-recursion without PIC can become rupture. Self-recursion without External Anchor can become self-justifying closure.

Thus, the formula is defensive. It defines what cannot be ignored.

E. Boundary 3: External Anchor Is Functional, Not Human-Essentialist

V7 Final requires External Anchor, but it defines the anchor by function rather than by metaphysical subject.

An External Anchor is any sufficiently independent source of contradiction, evaluation, or continuity-preserving constraint that prevents self-recursive collapse into self-justifying closure.

In the present implementation, human ownership is the main practical form of external anchoring. Human seat, approval, responsibility, release judgment, and stop judgment remain necessary.

However, V7 Final does not claim that humans are the only possible anchor forever. A future non-human structure could in principle serve the anchoring role if it satisfies the required properties: independence, tamper resistance, contradiction, evaluation, continuity preservation, and non-erasability by the self-recursive system.

The boundary is therefore:

Human ownership is the present implementation of external anchoring, not the only possible form.

This preserves current human decision authority without turning the paper into human-essentialism.

F. Boundary 4: Capability Growth Does Not Expand Authority

A recurring error in self-improving-system discourse is to infer authority from capability.

V7 Final rejects this inference.

Capability growth may make a system more useful, more general, or more powerful. It may also become material for future self-recursive updating. But capability growth does not expand the system's authority to bypass Guard, External Anchor, memory continuity, As-of responsibility, carrier preservation, or restartable completion.

A system that becomes more capable can still become less guardable.

Therefore:

Capability gain may become material for self-evolution, but it does not by itself authorize self-evolution.

This distinction is central. Without it, stronger systems would receive fewer constraints precisely when their updates become more consequential.

G. Boundary 5: C1–C9 Are a Coverage Map, Not an AGI Checklist

C1–C9 should not be read as a checklist that, once completed, certifies AGI.

They are a Failure Mode Coverage Map. Each condition identifies a failure that reopens when the condition is missing.

The map does not say:

If C1–C9 are satisfied, AGI exists.

It says:

If a self-recursive evolution claim ignores C1–C9, known failure terrain remains untreated.

This difference matters. A checklist certifies arrival. A coverage map exposes untreated failure.

V7 Final uses C1–C9 in the second sense only.

H. Boundary 6: Precondition Topology Is Not a Completion Map

The Precondition Topology does not show a completed AGI region.

It shows an entrance terrain. The central basin is not a domain of unconstrained capability expansion. It is a defensible region in which self-recursive evolution can be studied without leaving known failure regions untreated.

The topology therefore has a limited role:

It defines what must be resolved before self-recursive evolution can be safely treated as a research object.

It does not certify that evolution has occurred. It does not measure intelligence. It does not rank systems.

I. Boundary 7: Reconstructive Delta Is Not Proof of Evolution

Reconstructive Delta is an operational handle.

It identifies reconstruction error that may become material for future update if the difference is detected, stoppable, structurally identified, and admitted through Guardable Continuity.

But delta is not proof.

A difference can be noise, hallucination, drift, memory corruption, or self-justifying reinterpretation. Treating every difference as progress would destroy the defensive structure of V7 Final.

Therefore:

Reconstructive Delta may supply future update material, but it does not prove self-evolution.

The admissibility of a delta depends on whether it preserves Guardable Continuity and does not reopen the failure modes mapped by C1–C9.

J. Boundary 8: V7 Final Does Not Remove Future Research

V7 Final narrows the claim in order to keep it defensible. This narrowing should not be mistaken for closure.

The paper leaves many tasks open:

- formalizing public and private gates for evolution activation;
- specifying stronger empirical handles for Reconstructive Delta;
- testing failure-mode coverage under multi-model or multi-agent settings;
- refining non-human external anchoring candidates;
- studying when external anchoring can be safely reduced or replaced;
- connecting the topology to measurable long-horizon agent behavior.

These are future research directions. They are not required for the narrower contribution of this paper: defining the structural failure boundaries that must be addressed before guardable self-recursive evolution is claimed.

K. Final Boundary Statement

V7 Final does not solve self-evolution. It defines the structural gate that a self-recursive evolution claim must pass before it can be treated as guardable.

The paper does not claim to fully define AGI, does not treat $A \times G \times I$ as a sufficient condition for completed AGI, does not use C1–C9 as an AGI checklist, and does not treat Reconstructive Delta as proof of evolution. External Anchor is also defined functionally: human ownership is the present implementation of anchoring, not the only logically possible form.

The defensive contribution is narrower:

A system that claims guardable self-recursive evolution must explain why its update process does not reopen vacuous recursion, drift, rupture, self-justifying closure, hindsight rewriting, memory discontinuity, carrier collapse, or false completion.

V7 Final therefore defines the failure terrain that any later operational gate must not ignore.

This is the defensive contribution of the paper. The conclusion now returns to the central thesis and closes the manuscript.

X. Conclusion

V7 Final began from a limited question.

It did not ask how many benchmarks a system must pass before it is called AGI. It did not ask whether a system can produce more fluent outputs, solve more tasks, or imitate broader forms of human performance. Those questions remain important, but they are not the question of this paper.

The question of V7 Final is:

What structural failures must a self-recursive intelligence avoid before its evolution can be called guardable?

The answer developed in this paper is the Guardable Continuity Axiom:

Evolution is admissible only when change preserves guardable continuity.

This axiom separates change from evolution. A system may become more capable while becoming less guardable. It may update itself while losing direction, compress memory while destroying the path back, or close work while leaving the future self unable to restart. For V7 Final, such changes are not evolution. They are failure modes.

A. Summary of the Structural Gate

The paper defined Decision-OS as a gate-first operational architecture for preserving guardable continuity across decision, update, integration, memory, release, and completion.

It then applied this architecture to self-recursive intelligence through the Door Formula:

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (9)$$

The symbol “ $\times$ ” denotes logical meet, not numerical multiplication. Aspire supplies direction. Guard supplies admissibility boundaries. Intelligence is treated as self-recursion on PIC: the ability to update, reuse, and re-evaluate structure without collapsing into rupture, order dependence, or safety dilution.

This structure is further constrained by External Anchor. A closed self-recursive system can justify its own outputs, compress its own memory, and declare its own completion. Without a sufficiently independent source of contradiction, evaluation, and continuity-preserving constraint, it remains exposed to self-justifying closure.

Human ownership is the present implementation of external anchoring, not the only possible form. But current systems do not gain the authority to remove human seat, responsibility, approval, release judgment, or stop judgment merely by increasing capability.

B. Failure Boundary Contribution

The main contribution of V7 Final is the failure boundary.

The paper introduced C1–C9 as a Failure Mode Coverage Map. These are not sufficient conditions for AGI and not a checklist that certifies arrival. They identify which known failure modes reopen when each condition is missing.

If direction is missing, recursion becomes vacuous. If Guard is missing, update becomes unguardable. If PIC is missing, integration ruptures. If self-recursion is missing, the system remains external tuning. If trajectory is missing, point evaluation hides drift. If As-of responsibility is missing, the past is rewritten by hindsight. If carrier preservation is missing, Aspire consumes its own carrier. If reconnectable forgetting is missing, memory breaks the line. If completion integrity is missing, closure becomes false completion.

This is the structural gate of V7 Final.

C. Precondition Topology and Reconstructive Delta

The paper then reorganized C1–C9 into a Precondition Topology.

This topology does not draw a completed AGI region. It draws the entrance terrain that must be addressed before self-recursive evolution can be safely treated as a research object. Outside the basin are known failure regions: vacuous recursion, unguardable update, rupture, mere adaptation, trajectory damage, hindsight rewriting, carrier collapse, memory discontinuity, false completion, and self-justifying closure.

At the entrance is $A \times G \times I$. Around the basin are C1–C9. At the center is not completed AGI, but a guardable self-recursive basin: the minimum defensive region in which self-recursive evolution can be discussed without leaving known failure terrain untreated.

Finally, the paper introduced Reconstructive Delta. Reconstructive Delta does not prove evolution. It is an operational handle for detecting when reconstruction error may become material for future self-recursive updating.

The minimal conversion path is:

$$\text{Reconstruction Error} \rightarrow \text{Detect} \rightarrow \text{Stop} \rightarrow \text{Identify} \rightarrow \text{Delta} \rightarrow \text{Future Update}. \quad (10)$$

A difference becomes useful only when it is detected, stoppable, structurally identified, and admitted through Guardable Continuity. Otherwise, it remains error, drift, hallucination, memory corruption, or self-justifying reinterpretation.

D. Final Position

V7 Final does not solve AGI.

It does not prove sufficient conditions for self-evolution. It does not claim that current AI systems possess autonomous self-evolution authority. It does not make External Anchor equivalent to permanent human-essentialism. It does not turn C1–C9 into a certification checklist.

Its claim is narrower and harder:

A system that claims guardable self-recursive evolution must explain why its update process does not reopen vacuous recursion, drift, rupture, self-justifying closure, hindsight rewriting, memory discontinuity, carrier collapse, or false completion.

This is the gate.

A future self-recursive system may not cite Decision-OS. It may use different terminology, a different mathematical language, or a different implementation. But if it claims guardable self-recursive evolution, it must still solve the same failure modes.

V7 Final is therefore not a paper that declares AGI has arrived. It is a paper that defines the minimum structural gate from which guardable self-recursive evolution research can safely begin.

Author Responsibility and Disclosure

The author assumes sole responsibility for the research design, conceptual claims, terminology, scope boundaries, acceptance or rejection of model-generated alternatives, and all publication decisions in this manuscript.

AI systems were used as drafting, comparison, translation, compression, and language-refinement tools during the preparation of this work. All substantive decisions, including the selection of claims, the preservation of scope boundaries, and the final structure of the manuscript, were made by the human author.

This paper should be read as a conceptual and operational framework paper. It does not claim to implement a complete AGI system, a complete self-evolution mechanism, or an empirical validation of all proposed gates. Its contribution is to define the structural failure boundaries that must be addressed before guardable self-recursive evolution is claimed.

The Japanese source-state master for V7 Final is treated as the fixed source-state record. This English manuscript is a submission-facing compressed translation and reconstruction of that source-state master, not a literal sentence-by-sentence translation.

The Japanese source-state master is fixed at the following GitHub commit:

f6124e81c7cf1d87b6ea5dd89bb1a140d4567e10

Repository path:

notes/v7-final/

The Japanese source-state master is an internal source-state record for lineage preservation. The present English manuscript is the submission-facing paper.

References

- [1] R. Xu, Z. Wang, R.-Z. Fan, and P. Liu, “Benchmarking benchmark leakage in large language models,” arXiv preprint arXiv:2404.18824, 2024. [Online]. Available: <https://arxiv.org/abs/2404.18824>
- [2] J. Haimes, C. Wenner, K. Thaman, V. Tashev, C. Neo, E. Kran, and J. Schreiber, “Benchmark inflation: Revealing llm performance gaps using retro-holdouts,” arXiv preprint arXiv:2410.09247, 2024. [Online]. Available: <https://arxiv.org/abs/2410.09247>
- [3] M. Shapiro, N. Pregoça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Stabilization, Safety, and Security of Distributed Systems*, 2011.
- [4] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” arXiv preprint arXiv:2310.08560, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [5] L. Moreau and P. Missier, “Prov-dm: The prov data model,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [6] B. Fischhoff, “Hindsight is not equal to foresight: The effect of outcome knowledge on judgment under uncertainty,” *Journal of Experimental Psychology: Human Perception and Performance*, vol. 1, no. 3, pp. 288–299, 1975.
- [7] National Institute of Standards and Technology, “Artificial intelligence risk management framework (ai rmf 1.0),” U.S. Department of Commerce, National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [8] S. Nagata, “Decision-os v4: Polaris-origin,” 2026, decision-OS research sequence.
- [9] —, “Decision-os v5 revised (siriusa2): A zero-knowledge confirmation layer for trajectory-aware human-ai decision safety,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19828435>
- [10] —, “Decision-os v6: Canonical commitment architecture for non-destructive integration,” 2026, decision-OS research sequence.
- [11] —, “Decision-os v7 addendum: Why aspire and why pic,” 2026, decision-OS research sequence.
- [12] —, “Decision-os v8: Time-tube control for self-safe agi,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19690553>
- [13] —, “Decision-os v9.1: Impact-weighted release — from continuous-pass gates to condition-bound judgment reuse,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19935535>
- [14] —, “Decision-os v10: Survival-bounded planning for aspire-preserving ai assistance,” 2026, decision-OS research sequence. [Online]. Available: <https://doi.org/10.5281/zenodo.20371623>
- [15] —, “Decision-os v11: Forget for future — reconnectable forgetting for long-horizon agentic ai,” 2026, decision-OS research sequence. [Online]. Available: <https://doi.org/10.5281/zenodo.20301056>
- [16] —, “Decision-os v12: Completion integrity,” 2026, decision-OS research sequence. [Online]. Available: <https://doi.org/10.5281/zenodo.20102241>